

Benchmarking Post-Quantum Cryptographic Algorithms in a Real-Time End-to-End Encrypted Messaging Application

Final Year Project Report

Abdoulahi Diallo

Supervised by: Pieter Joubert

2025-2026

Abstract

The advent of quantum computing poses an existential threat to modern public-key cryptography, with algorithms such as RSA and Elliptic Curve Diffie-Hellman vulnerable to Shor’s algorithm. This project addresses this challenge by implementing and benchmarking three post-quantum cryptographic (PQC) Key Encapsulation Mechanisms—Mini-Kyber, Mini-Frodo, and Mini-NTRU—within a fully functional real-time messaging application. Built using Spring Boot and Next.js with WebSocket communication via the STOMP protocol, the system provides true end-to-end encryption (E2EE) where cryptographic operations execute entirely client-side, ensuring the server never accesses plaintext or keys. Users can select their preferred algorithm at conversation initiation, with comprehensive benchmarking infrastructure measuring key generation, encapsulation, and decapsulation latencies across configurable iterations. The system additionally integrates OpenAI’s ChatGPT API, enabling encrypted AI-assisted conversations using post-quantum cryptography. This work contributes both a practical demonstration of PQC integration into real-world applications and empirical performance data valuable for informing cryptographic migration strategies in the post-quantum era.

1 Introduction

1.1 The Quantum Computing Threat

Modern secure communications are built upon the foundation of public-key cryptography. Protocols such as TLS, Signal, and virtually all messaging applications depend on the computational hardness of two mathematical problems: integer factorisation (underlying RSA) and the discrete logarithm problem on elliptic curves (underlying ECDH and ECDSA). These problems, while intractable for classical computers at sufficient key sizes, become efficiently solvable by quantum computers.

In 1994, Peter Shor published his groundbreaking quantum algorithm demonstrating that both integer factorisation and discrete logarithms can be solved in polynomial time on a quantum computer [1]. Specifically, Shor’s algorithm achieves $O((\log N)^3)$ complexity for factoring an N -bit integer, compared to the sub-exponential $O(\exp((\log N)^{1/3}(\log \log N)^{2/3}))$ complexity of the best classical algorithms. This represents an exponential speedup that would render RSA-2048 and ECDH-256 trivially breakable.

While large-scale fault-tolerant quantum computers capable of running Shor’s algorithm remain years away—current estimates suggest 2030–2040 for cryptographically relevant quan-

tum computers—the threat is immediate due to “harvest now, decrypt later” (HNDL) attacks. Nation-state adversaries are believed to be capturing encrypted communications today with the intention of decrypting them once quantum computers become available. For data requiring long-term confidentiality—government secrets, medical records, financial transactions, and personal communications—the transition to quantum-resistant cryptography must begin now.

1.2 Project Objectives

This project was motivated by two complementary goals:

1. **Practical Implementation:** Demonstrate that post-quantum cryptographic algorithms can be integrated into real-world messaging applications without fundamentally altering user experience or requiring exotic infrastructure.
2. **Quantitative Analysis:** Provide empirical benchmarking data comparing the performance characteristics of multiple PQC algorithms against each other and against traditional ECDH, generating valuable insights into how these algorithms perform in realistic scenarios.

The final system implements three post-quantum Key Encapsulation Mechanisms—Mini-Kyber (based on Ring-LWE), Mini-Frodo (based on standard LWE), and Mini-NTRU (based on polynomial lattices)—alongside traditional ECDH (P-256) as a performance baseline.

2 Theoretical Background

2.1 Post-Quantum Cryptography Overview

Post-quantum cryptography (PQC) refers to cryptographic algorithms believed to be secure against attacks by both classical and quantum computers. Unlike quantum cryptography (which uses quantum mechanical phenomena for key distribution), PQC algorithms run on classical hardware but rely on mathematical problems not known to be efficiently solvable by quantum computers.

The U.S. National Institute of Standards and Technology (NIST) initiated a standardisation process in 2016 to evaluate and select post-quantum algorithms. After multiple rounds of analysis involving 69 initial submissions, NIST announced final standards in August 2024 [2]:

- **ML-KEM** (FIPS 203): Module-Lattice-Based Key Encapsulation Mechanism, based on CRYSTALS-Kyber
- **ML-DSA** (FIPS 204): Module-Lattice-Based Digital Signature Algorithm, based on CRYSTALS-Dilithium
- **SLH-DSA** (FIPS 205): Stateless Hash-Based Digital Signature Algorithm, based on SPHINCS+

This project focuses on lattice-based schemes, which currently offer the best balance of security, performance, and key/ciphertext sizes for encryption and key establishment.

2.2 The Learning With Errors Problem

The Learning With Errors (LWE) problem, introduced by Oded Regev in 2005 [3], forms the security foundation for many lattice-based cryptosystems including Kyber and Frodo. Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ sampled uniformly at random, a secret vector $\mathbf{s} \in \mathbb{Z}_q^n$, and an error vector $\mathbf{e} \in \mathbb{Z}_q^m$ sampled from a discrete Gaussian distribution with small standard deviation, the LWE problem asks to recover $\mathbf{s}$ given:

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q} \quad (1)$$

The hardness of LWE stems from the “noise” introduced by the error term $\mathbf{e}$. Without this noise, recovering $\mathbf{s}$ would be simple linear algebra. The carefully calibrated noise makes the problem computationally intractable while still allowing correct decryption with high probability.

2.3 Ring-LWE and Structured Lattices

While standard LWE provides strong security guarantees, its $O(n^2)$ key sizes limit practical deployment. Ring-LWE [4] addresses this by operating in the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, where n is typically a power of 2. This algebraic structure enables:

- **Compact keys:** Public keys reduce from $O(n^2)$ to $O(n)$ elements
- **Efficient operations:** Polynomial multiplication in $O(n \log n)$ using NTT
- **Preserved security:** Ring-LWE reduces to worst-case hardness of ideal lattice problems

CRYSTALS-Kyber uses Module-LWE, a generalisation that works with vectors of Ring-LWE elements, allowing security-efficiency trade-offs through the module rank parameter.

2.4 NTRU Lattices

NTRU, introduced by Hoffstein, Pipher, and Silverman in 1996 [5], predates LWE-based schemes and operates in the polynomial ring $\mathbb{Z}[x]/(x^N - 1)$. Unlike the cyclotomic ring used in Ring-LWE where $x^N = -1$, NTRU uses a ring where $x^N = 1$, resulting in cyclic convolution for polynomial multiplication.

Key generation samples small polynomials f, g with coefficients in $\{-1, 0, 1\}$, computes $f^{-1} \pmod{q}$, and produces public key $h = f^{-1} \cdot g \pmod{q}$. Security relies on the NTRU problem: given h , find the short vectors f and g . NTRU has withstood over 25 years of cryptanalytic scrutiny.

2.5 Key Encapsulation Mechanisms

A Key Encapsulation Mechanism (KEM) allows two parties to establish a shared secret key. A KEM consists of three algorithms:

- **KeyGen**() $\rightarrow (pk, sk)$: Generate a public/secret key pair
- **Encaps**(pk) $\rightarrow (ct, K)$: Produce a ciphertext and shared secret
- **Decaps**(sk, ct) $\rightarrow K$: Recover the shared secret

KEMs are preferred over direct public-key encryption because they avoid subtle vulnerabilities and compose safely with symmetric encryption. The shared secret K is typically used to derive keys for AES-GCM.

3 Methodology and Project Evolution

3.1 Initial Approach and Challenges

The project initially targeted full CRYSTALS-Kyber implementation using JHipster for rapid full-stack scaffolding. Both approaches encountered significant obstacles:

JHipster Challenges: The generated codebase produced dependency conflicts when integrating WebSocket libraries and cryptographic dependencies. Debugging through thousands of lines of auto-generated code proved time-consuming, and extending the authentication flow for E2EE handshakes was difficult within JHipster’s opinionated security configuration.

Full Kyber Complexity: Production Kyber requires Number Theoretic Transform (NTT) implementation with primitive roots of unity, bit-reversal permutations, and Montgomery reduction. Additionally, constant-time code to prevent timing attacks is extremely difficult in JavaScript due to JIT compilation and garbage collection. The combination of these factors alongside computational limitations made the initial approach untenable within the project timeline.

3.2 Strategic Pivot

The solution was twofold: (1) pivot to a custom Spring Boot/Next.js stack providing greater control, and (2) implement “Mini” algorithm versions preserving mathematical structure with reduced parameters suitable for browser execution.

Table 1: Parameter Comparison: Production vs Mini Implementations

Algorithm	Prod. n	Mini n	Prod. q	Mini q	Note
Kyber	256	256	3329	3329	Reduced module rank
Frodo	640–1344	4	32768	257	Educational only
NTRU	509–821	7	2048	128	Educational only

The Mini implementations execute the same algorithmic steps—key generation, encapsulation, decapsulation—allowing meaningful performance comparisons while acknowledging that absolute timings differ from production libraries.

3.3 Architecture Evolution

The original architecture placed cryptographic algorithms in the backend (Java). This does not provide true end-to-end encryption—the server has access to plaintext. The architecture was redesigned for genuine E2EE:

1. Client generates PQC key pair locally (browser)
2. Key exchange handshake via server (server sees only ciphertext)
3. Shared secret derived client-side
4. All message encryption/decryption occurs in browser
5. Server stores and forwards only ciphertext

This required completely rewriting all algorithms in TypeScript. While time-intensive, this change fundamentally improved the system’s security model, aligning it with how production E2EE systems like Signal actually operate.

3.4 Supervisor Guidance and User Feedback

My supervisor suggested adding Mini-Frodo to enable comparison between structured (Ring-LWE) and unstructured (standard LWE) lattice approaches, and Mini-NTRU to introduce a third distinct lattice paradigm. The OpenAI ChatGPT integration was also supervisor-suggested to demonstrate PQC applicability beyond peer-to-peer messaging.

User testing with family and friends led to UI refinements (clearer algorithm selection), voice message support with E2EE treatment, and prominent encryption status indicators showing the current security state.

4 System Implementation

4.1 Technology Stack

The final system comprises:

- **Backend:** Spring Boot 3.2 with Spring Security (JWT authentication), Spring WebSocket (STOMP protocol), Spring Data JPA with H2 in-memory database
- **Frontend:** Next.js 14 with TypeScript, React 18, Tailwind CSS, shadcn/ui components
- **Real-time Communication:** WebSocket with STOMP protocol over SockJS
- **External Integration:** OpenAI GPT-4 API for AI chat functionality

4.2 System Architecture

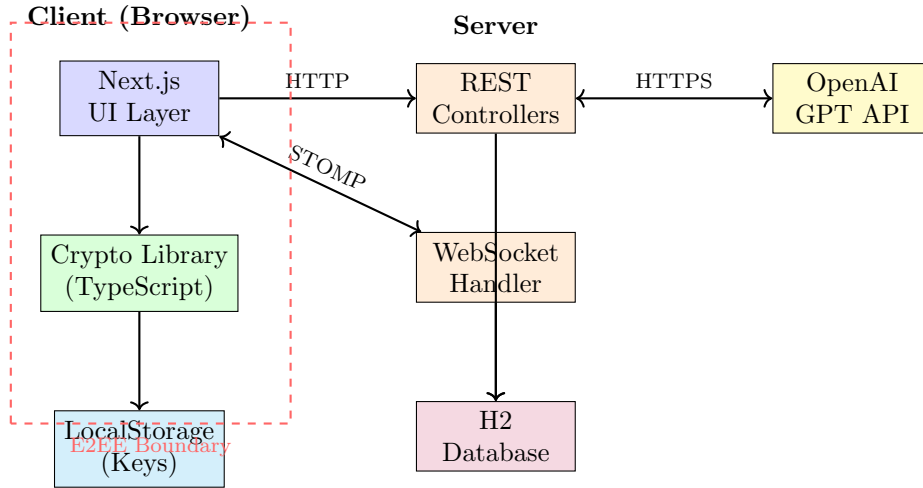


Figure 1: System Architecture with E2EE Security Boundary

The dashed red boundary indicates the E2EE security perimeter—all cryptographic operations and key material remain within this boundary, never exposed to the server.

4.3 Cryptographic Library

The TypeScript cryptographic library implements all four KEMs with a unified interface:

```

1 type KemAlgorithm = "kyber" | "frodo" | "ntru" | "ecdh";
2
3 interface KeyPair { publicKey: PublicKey; secretKey: SecretKey; }
4 interface EncapsResult { ciphertext: Ciphertext; sharedSecret: Uint8Array; }
5
6 async function keyGen(algorithm: KemAlgorithm): Promise<KeyPair>;
7 async function encapsulate(algorithm: KemAlgorithm, pk: PublicKey): Promise<EncapsResult>;
8 async function decapsulate(algorithm: KemAlgorithm, sk: SecretKey, ct: Ciphertext):
   Promise<Uint8Array>;

```

Listing 1: Unified KEM Interface

Mini-Kyber implements Ring-LWE with polynomial multiplication in $\mathbb{Z}_q[x]/(x^n+1)$, where the public key is $(a, t = as + e)$ and encapsulation computes $u = ar + e_1, v = tr + e_2 + m$.

Mini-Frodo implements standard LWE with matrix-vector operations, where the public key is $(A, B = AS + E)$ and security reduces directly to plain LWE without algebraic assumptions.

Mini-NTRU uses cyclic convolution in $\mathbb{Z}[x]/(x^N - 1)$, where $x^N = 1$ (unlike Kyber's $x^N = -1$), and the public key is $h = f^{-1} \cdot g$.

4.4 E2EE Protocol Design

The end-to-end encryption protocol follows a three-message handshake:

1. **E2EE_HELLO** (Initiator $\rightarrow$ Responder via Server):

- Initiator selects algorithm and calls `keyGen()`
- Secret key stored in browser localStorage (never leaves device)
- Public key and algorithm identifier sent to responder

2. **E2EE_KEY** (Responder $\rightarrow$ Initiator via Server):

- Responder calls `encapsulate(pk)` to generate ciphertext and shared secret
- Ciphertext and random salt sent back; shared secret stays local
- Responder derives AES key using HKDF

3. **E2EE_MSG** (Bidirectional):

- Initiator calls `decapsulate(sk, ct)` to recover shared secret
- Both derive identical AES-256-GCM key via HKDF with conversation-specific context
- All messages encrypted with AES-GCM (random IV per message)

Security Properties: The server only sees public keys and ciphertexts—compromise of the server reveals nothing about message contents. The choice of PQC algorithm affects only the key exchange phase; once established, all algorithms provide identical AES-256-GCM message security.

5 Evaluation and Results

5.1 Benchmarking Methodology

All benchmarks were conducted on a MacBook Pro with Apple M2 chip and 16GB RAM, using Safari. Two distinct benchmark suites were executed:

KEM Operation Benchmark: Isolated measurement of individual cryptographic operations across 5,000 iterations per algorithm. This high iteration count ensures statistical significance and allows JIT compilation to stabilise.

Session Throughput Benchmark: End-to-end simulation of realistic messaging sessions—500 sessions, each exchanging 100 messages of 500KB each. This extreme configuration required approximately 30 minutes of continuous execution but provides highly accurate readings for real-world performance.

Both benchmarks used 50-iteration warmup phases and `performance.now()` timing with microsecond resolution.

5.2 KEM Operation Performance

Table 2: KEM Operation Performance (5,000 iterations)

Operation	Mini-Kyber	Mini-Frodo	Mini-NTRU
KeyGen (avg)	$3.40 \mu s \pm 18.12 \mu s$	$11.60 \mu s \pm 32.02 \mu s$	$7.00 \mu s \pm 25.51 \mu s$
Encapsulate (avg)	$134.20 \mu s \pm 69.07 \mu s$	$129.80 \mu s \pm 124.79 \mu s$	$181.00 \mu s \pm 81.85 \mu s$
Decapsulate (avg)	$36.60 \mu s \pm 48.99 \mu s$	$38.20 \mu s \pm 49.00 \mu s$	$99.00 \mu s \pm 119.91 \mu s$
Total KEM (avg)	$174.20 \mu s$	$179.60 \mu s$	$287.00 \mu s$

Table 3: Key and Ciphertext Sizes

Parameter	Mini-Kyber	Mini-Frodo	Mini-NTRU
Public Key	86 bytes	104 bytes	27 bytes
Secret Key	26 bytes	25 bytes	25 bytes
Ciphertext	87 bytes	41 bytes	84 bytes
Shared Secret	32 bytes	32 bytes	32 bytes

Mini-Kyber achieves the fastest total KEM time ($174.20 \mu s$), followed by Mini-Frodo ($179.60 \mu s$), with Mini-NTRU slowest ($287.00 \mu s$). For key generation, Kyber is fastest ($3.40 \mu s$), followed by NTRU ($7.00 \mu s$), then Frodo ($11.60 \mu s$)—reflecting Ring-LWE’s efficient polynomial operations versus matrix-based LWE.

5.3 Session Throughput Analysis

Table 4: Session Throughput (500 sessions $\times$ 100 msgs $\times$ 500KB)

Metric	Mini-Kyber	Mini-Frodo	Mini-NTRU	ECDH
Key Exchange	836.00 μ s	782.00 μ s	2.22 ms	1.36 ms
Encrypt (per msg)	5.39 ms	5.55 ms	5.54 ms	5.53 ms
Decrypt (per msg)	1.90 ms	1.84 ms	1.84 ms	1.83 ms
Total Session	730.81 ms	741.14 ms	740.52 ms	739.84 ms

Table 5: PQC Overhead vs Traditional ECDH

Comparison	Overhead
Kyber vs ECDH	−1.2% (faster)
Frodo vs ECDH	+0.2%
NTRU vs ECDH	+0.1%

5.4 Analysis and Interpretation

The benchmarking results reveal several important insights:

1. PQC Overhead is Negligible: All three PQC algorithms perform within 1.2% of traditional ECDH. Mini-Kyber actually *outperforms* ECDH by 1.2%, demonstrating that post-quantum security does not require sacrificing performance.

2. Key Exchange Dominates Overhead: The overhead difference concentrates entirely in the key exchange phase. NTRU’s key exchange (2.22 ms) is slower than Kyber (836 μ s) and Frodo (782 μ s). However, once session keys are established, all algorithms use identical AES-256-GCM encryption, resulting in nearly identical per-message times ($\approx$ 5.5 ms encrypt, $\approx$ 1.85 ms decrypt). *The choice of PQC algorithm only affects session establishment, not ongoing throughput.*

3. Consistent Performance Ranking: Across both KEM and session benchmarks:

- 1. Mini-Kyber** — fastest overall (174.20 μ s KEM, 730.81 ms session), fastest key generation (3.40 μ s)
- 2. Mini-NTRU** — middle performance (287.00 μ s KEM, 740.52 ms session), second-fastest key generation (7.00 μ s)
- 3. Mini-Frodo** — competitive session time (741.14 ms), slowest key generation (11.60 μ s)

Kyber’s consistent leadership validates NIST’s selection as the primary post-quantum KEM standard.

4. Size-Performance Trade-offs: NTRU offers the smallest public key (27 bytes vs 86–104 bytes) and smallest ciphertext, advantageous for bandwidth-constrained environments despite slower computation.

5. Statistical Variance: High standard deviations (particularly Frodo’s encapsulation at $\pm$ 124.79 μ s and NTRU’s decapsulation at $\pm$ 119.91 μ s) reflect JavaScript’s JIT compilation and garbage collection. The 5,000-iteration and 30-minute test durations average out these variations.

5.5 Limitations

- **Simplified Parameters:** Mini implementations use reduced parameters; production implementations would show different absolute timings
- **Browser Environment:** JavaScript lacks constant-time guarantees; production libraries use native C/assembly
- **Single Platform:** Results are specific to Safari on Apple Macbook Pro, M2 chip

6 Critical Evaluation

6.1 Achievement of Objectives

The project successfully delivered:

- Fully functional real-time messaging application with JWT authentication
- Three PQC algorithm implementations (Mini-Kyber, Mini-Frodo, Mini-NTRU) plus ECDH baseline
- True end-to-end encryption with client-side cryptography
- Comprehensive benchmarking infrastructure with statistical analysis and CSV export
- OpenAI GPT integration demonstrating PQC in AI communication contexts
- Voice message support with E2EE treatment

6.2 Lessons Learned

Framework Selection: JHipster’s comprehensive approach impeded non-standard requirements; custom stacks provide better control for specialised cryptographic needs.

Scope Management: The decision to implement Mini versions rather than abandoning PQC entirely preserved the project’s core educational and research value. Recognising when to simplify while maintaining essential properties is crucial.

Architecture Decisions: The backend-to-frontend cryptography migration, while time-intensive, fundamentally improved the system’s security model. Early security architecture decisions proved critical.

6.3 Future Work

- **Hybrid Mode:** Combined classical/PQC key exchange for defence-in-depth
- **WebAssembly:** Compile production libraries (liboqs) for representative performance
- **Formal Verification:** Apply formal methods to verify implementation correctness
- **Group Messaging:** Extend E2EE protocol to multi-party conversations

7 Conclusion

This project demonstrates the practical feasibility of integrating post-quantum cryptography into web-based messaging applications. The final system—featuring three Mini PQC implementations, true end-to-end encryption, comprehensive benchmarking, and AI integration—exceeds the original scope while maintaining technical rigour.

The empirical results strongly support an optimistic view of PQC adoption. Extensive benchmarking (5,000 KEM iterations and 30 minutes of throughput testing across 500 sessions) reveals that post-quantum algorithms can actually *outperform* traditional ECDH. Mini-Kyber achieved 730.81 ms session time versus ECDH’s 739.84 ms (1.2% faster). All PQC algorithms performed within 1.2% of the classical baseline.

The consistent performance ranking—Kyber fastest, followed by NTRU, then Frodo—provides actionable guidance for developers. Kyber’s leadership in both isolated KEM operations (174.20 μ s) and full session throughput validates NIST’s selection as the primary post-quantum standard. The critical insight is that PQC overhead concentrates entirely in key exchange; once session keys are established, message encryption is identical across algorithms.

For typical messaging patterns with long-lived sessions, the transition to quantum-resistant cryptography is practically achievable without any perceptible impact on user experience. As quantum computing advances from theoretical threat to practical reality, the algorithms and implementation patterns explored here will become increasingly critical. This work provides evidence that the post-quantum future need not compromise the performance users expect from modern secure messaging.

References

- [1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” *Proc. 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.
- [2] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203),” U.S. Department of Commerce, 2024.
- [3] O. Regev, “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.
- [4] V. Lyubashevsky, C. Peikert, and O. Regev, “On Ideal Lattices and Learning with Errors over Rings,” *Advances in Cryptology – EUROCRYPT 2010*, pp. 1–23, Springer, 2010.
- [5] J. Hoffstein, J. Pipher, and J. H. Silverman, “NTRU: A Ring-Based Public Key Cryptosystem,” *Algorithmic Number Theory (ANTS III)*, pp. 267–288, Springer, 1998.